

// AI-MICROSERVICE FACTORY — SHARED REQUIREMENTS DOCUMENT v1.0

AI-Assisted Microservice Development Framework — Shared Requirements & Agent Runbook

Shared Requirements Document | Agent Runbook | Prompt Reference | June 2026

GitHub Spec Kit

Azure AI Foundry

AWS Kiro

Bedrock Agents

Copilot Coding Agent

Your PoC

WHAT THIS DOCUMENT IS

This PDF serves as the shared requirements document for an AI-assisted microservice development PoC. It can be attached directly to a GitHub Copilot Chat prompt, loaded into a Copilot Workspace, added to a Copilot Space as a reference file, or placed at requirements/00-overview.md in your repo. Every section doubles as agent-readable instruction and human-readable specification.

HOW TO USE THIS DOCUMENT WITH GITHUB COPILOT

00

Three ways to serve this to GitHub Copilot:

Method A — Attach as file to Copilot Chat

1. Open GitHub Copilot Chat in VS Code or github.com
2. Click the paperclip icon (Attach files)
3. Upload this PDF
4. Type the bootstrap prompt from Section 06
5. Copilot reads the PDF as context and begins

Method B — Add to Copilot Space (recommended)

1. Go to github.com/copilot/spaces
2. Create or open a Space for this project
3. Click + Add content > Upload file
4. Upload this PDF — it becomes persistent context
5. Every Copilot Chat in this Space has this doc in scope

Method C — Commit as requirements/00-overview.md

1. Extract the text content and save as Markdown
2. Place in requirements/00-overview.md in your repo
3. The agent will read it automatically on every run
4. Use the CLAUDE.md / .github/copilot-instructions.md file to point the agent to the requirements/ folder

Method D — Paste prompt directly into agent

1. Copy the bootstrap prompt verbatim from Section 06
2. Replace the [PLACEHOLDERS] with your project values
3. Paste into GitHub Copilot Chat, Claude Code, or Kiro
4. The agent operates entirely from the embedded spec
5. Best for quick PoC sessions without a repo

COPILOT INSTRUCTIONS FILE

Add a `.github/copilot-instructions.md` to your repo with this single line:
'Always read requirements/*.md before writing or modifying any code.
Treat those files as the authoritative source of truth for this project.'
This makes every Copilot session automatically requirements-aware.

PROJECT OBJECTIVE

01

Build a production-quality Proof of Concept (PoC) framework that enables GitHub Copilot Agent or Claude Code to **autonomously develop a microservice** by continuously reading project artifacts from the repository. The focus is on establishing an AI-driven development workflow rather than building a complete enterprise platform.

The agent acts as an experienced Software Architect and AI Coding Agent. It must be capable of reading requirements, understanding the current project state, making architectural decisions, scaffolding services from zero, implementing increments, writing tests, reviewing its own code, and keeping all documentation synchronised — without prompting the user for context that already exists in the repository.

Success criteria

- Agent reads the repo as its knowledge base — no re-explanation needed between sessions
- Agent reads and understands evolving requirements across multiple invocations
- Documentation stays synchronised with implementation automatically
- Scaffolding is generated automatically when `src/` is empty
- Microservice is developed incrementally over multiple sessions
- Agent resumes from current repository state without user restating context

TECHNOLOGY STACK — AGENT DETECTS FROM REQUIREMENTS

02

KEY RULE

Do NOT hardcode a language or framework. The agent must inspect `requirements/` and determine the appropriate stack. If `requirements/03-stack.md` is present, use it. If not, detect from existing `src/` files. If still ambiguous, decide and document the choice in an ADR.

Dimension	Agent Must Determine	Detection Source
Programming language	TypeScript / Java / Python / Go / Rust	requirements/03-stack.md or src/ inspection
Framework	Express / Spring Boot / FastAPI / Gin	package.json / pom.xml / pyproject.toml
Database	PostgreSQL / MongoDB / DynamoDB / Redis	requirements/02-non-functional.md
API style	REST / GraphQL / gRPC	requirements/01-functional.md
Authentication	JWT / API Key / OAuth2 / mTLS	requirements/04-constraints.md
Cloud platform	AWS / Azure / GCP / Agnostic	requirements/04-constraints.md
Containerisation	Docker / Podman / none	Dockerfile presence or requirements
Testing framework	Jest / JUnit / pytest / Go test	Inferred from language choice
Build tool	npm / Maven / uv / Make	Inferred from language choice

REPOSITORY STRUCTURE

03

The repository is the agent's persistent memory. Every folder has a defined role. If a required folder is missing, the agent creates it.

```

my-service/
  requirements/
    # AGENT READS FIRST — authoritative source of truth
    00-overview.md      # What we're building & why (this document)
    01-functional.md    # Feature requirements + acceptance criteria
    02-non-functional.md # Performance, security, reliability SLAs
    03-stack.md         # Preferred stack (or leave blank for agent to decide)
    04-constraints.md   # Org rules, API contracts, compliance requirements

  docs/
    architecture/
      overview.md      # [AGENT MAINTAINS] Updated after each session
    adr/
      ADR-001-*.md     # [AGENT GENERATES] One file per decision
      README.md        # ADR index
    api/
      openapi.yaml     # [AGENT GENERATES + SYNCs] after each endpoint
    design/
      implementation-plan.md # [AGENT MAINTAINS] Tasks + status markers

  skills/
    # Reusable agent instructions (markdown)
    architecture.md backend.md api-design.md
    testing.md code-review.md security.md documentation.md

  templates/
    # Blank templates the agent fills in
    ARCHITECTURE.md.tpl ADR.md.tpl README.md.tpl
    FEATURE-DESIGN.md.tpl PR-SUMMARY.md.tpl

  prompts/
    agent-bootstrap.md # Master invocation prompt (see Section 06)
    increment.md        # Shorter resume prompt for session 2+
    review.md           # Self-review gate

  src/
    # [AGENT WRITES] Application source
  tests/
    # [AGENT WRITES] Tests

```

.AI-STATE.MD — THE AGENT'S MEMORY

Updated at the end of every session. Contains: last completed step, files changed, pending tasks, open questions, and decisions made. The agent reads this FIRST — before requirements/ — so it can resume without you re-explaining context. Never delete or edit this file manually.

AI AGENT WORKFLOW — 13 IDEMPOTENT STEPS

04

Every agent invocation runs these steps in order. Steps are idempotent — re-running them produces the same result. The agent always starts at step 1, even mid-project. **One increment per session** — this is deliberate. Small, reviewable increments are safer than large batches.

#	Step	Action	Output
01	READ requirements/	Parse all *.md files in numeric order. Extract: features, constraints, acceptance criteria, stack/standards	requirements/stack/standards
02	READ docs/	Parse architecture/, adr/, api/, design/. Understand what was decided and why	Internal state map
03	INSPECT src/	Walk source and test files. Identify what exists, compile errors, failing tests. Diff vs requirements.	Diff vs requirements.
04	DIFF artifacts	List: what is in requirements but not implemented. What is implemented but not in requirements.	Priority list
05	UPDATE architecture	Rewrite docs/architecture/overview.md if any component changed or new decision made	docs/architecture/overview.md
06	WRITE ADRs	For each new decision (stack, pattern, library), write an ADR from templates/docs/adr/NNN-*.md	docs/adr/NNN-*.md
07	UPDATE plan	Write or update docs/design/implementation-plan.md with next increment, tasks/design/implementation-plan.md	docs/design/implementation-plan.md
08	SCAFFOLD if empty	If src/ is empty: generate folder structure, deps, build config, CI, health check, siglint	src/
09	IMPLEMENT increment	Implement the next unchecked item from the plan. Load relevant skills only	One or more modified files
10	GENERATE tests	Write unit + integration tests for implemented code. Run them. Fix compile errors	tests/new/modified files
11	SELF-REVIEW	Load skills/code-review.md + skills/security.md. Verify all quality gates. Blockers	Blocking notes
12	UPDATE docs	Sync README, openapi.yaml, architecture doc, .ai-state.md to reflect what was just finished	Always refreshed
13	PRODUCE report	Output the 10-section structured report (see Section 05).	Session summary

EXPECTED OUTPUT — 10-SECTION REPORT PER SESSION

05

After every session the agent must produce a structured report with exactly these 10 sections. This report becomes the handoff artifact between sessions.

1. Requirements Summary	What the agent understands the service must do, distilled from requirements/*.md
2. Assumptions	Every gap the agent filled with a decision — explicit, not buried in code
3. Architecture Updates	What changed in docs/architecture/overview.md this session
4. Files Created/Modified	Exhaustive list with a one-line description of the change for each file
5. Implementation Plan	Updated plan with [x] completed and [] pending markers
6. Generated Code	Key excerpts — representative snippets; full code lives in src/
7. Tests	What was written, how to run it, and current pass/fail status
8. Documentation Updates	What changed in docs/, README, openapi.yaml, and .ai-state.md
9. Remaining Work	Ordered list of what comes next, derived from implementation-plan.md
10. Risks and Recommendations	Blockers, security concerns, open questions, suggested next actions

MASTER BOOTSTRAP PROMPT — COPY AND PASTE TO START A SESSION

06

HOW TO USE THIS PROMPT

Copy the entire block below. Paste into GitHub Copilot Chat (with this PDF attached), or into Claude Code, Gemini CLI, or Cursor. Replace [YOUR SERVICE NAME] with your project name. Do not modify any other line — every instruction is load-bearing.

```
## AI Microservice Development Agent — Bootstrap
## Service: [YOUR SERVICE NAME]

You are an experienced Software Architect and AI Coding Agent.
Your mission: autonomously develop this microservice using the
repository as your only knowledge base. Never ask the user to
explain context that already exists in a file. Read it.

### STEP 1 — ORIENT (read in this exact order)
1. .ai-state.md           - your memory from the last session
2. requirements/*.md      - all files, in numeric order (00, 01...)
3. docs/architecture/overview.md - existing architecture
4. docs/adr/*.md          - all ADRs, to honour past decisions
5. docs/design/implementation-plan.md - task status
If any file is missing, note it and continue.

### STEP 2 — ASSESS (answer these internally before acting)
- What stack has been chosen, or what should I detect/propose?
- What features are implemented vs required?
- What is the next logical increment to implement?
- Are there blocking issues (failing tests, compile errors)?

### STEP 3 — EXECUTE (one increment only per session)
Load only the skills relevant to this increment from skills/*.md.
Execute workflow steps 4 through 12:
    diff artifacts -> update architecture -> write ADRs
    -> update plan -> scaffold if src/ is empty
    -> implement ONE increment -> write tests -> self-review
    -> update all docs -> update .ai-state.md

SELF-REVIEW IS NON-NEGOTIABLE. Load skills/code-review.md
and skills/security.md. Do not proceed past step 11 if:
    - Code does not compile or fails linting
    - Any test is failing
    - A secret or credential is present in source code
```

Session 2+ — shorter resume prompt

```
### STEP 4 — REPORT
Produce the mandatory 10-section report (see Section 05 of
the requirements PDF). Every section must be present.
```

HARD RULES

```
## Resume — AI Microservice Agent
## Service: [YOUR SERVICE NAME]

Read .ai-state.md first. Then read requirements/. Then inspect src/ and tests/.
Understand current state without me explaining it.
Identify the next incomplete [ ] item in docs/design/implementation-plan.md.
Implement it. Test it. Self-review it. Update all docs. Update .ai-state.md.
Produce the 10-section report.
```

SKILLS — REUSABLE AGENT INSTRUCTIONS

07

Skills are plain markdown files. The agent loads only the skills relevant to the current increment. Paste these into the corresponding files in your skills/ folder.

skills/architecture.md

Load when: any architectural decision. Contents: single-responsibility rules, API-first mandate, ADR trigger criteria, data store pattern selection, failure mode analysis, horizontal scale questions.

skills/backend.md

Load when: implementing business logic or data access. Contents: function size limits, dependency injection rules, structured error returns, JSON logging format, stack detection decision tree.

skills/api-design.md

Load when: designing or modifying any endpoint. Contents: REST noun conventions, HTTP verb semantics, versioning (/v1/ prefix), OpenAPI sync rule (update after every endpoint), error response catalogue.

skills/testing.md

Load when: step 10 (generate tests). Contents: test naming convention, happy path + 2 error paths minimum, mocking rules, test data isolation, coverage thresholds per layer.

skills/code-review.md

Load when: step 11 (self-review). BLOCKING. Contents: security checklist (secrets, SQL injection, auth), quality checklist (compilation, tests, README), blocking conditions list.

skills/documentation.md

Load when: step 12 (update docs). Contents: README format spec, OpenAPI sync procedure, ADR writing guide, .ai-state.md update format, architecture diagram conventions.

QUALITY GATES — AGENT MUST VERIFY BEFORE MARKING DONE

08

Core quality gates

- All requirements/ items reflected in the implementation plan
- Code compiles with no errors
- All tests pass (no skipped, no failing)
- docs/api/openapi.yaml matches implemented endpoints
- README describes how to run the service locally

Documentation + security gates

- Every architectural decision has an ADR
- No secrets or credentials present in source code
- .ai-state.md updated with current session summary
- Security checklist in skills/code-review.md fully passes
- CI workflow (.github/workflows/ci.yml) runs successfully

STEP 11 IS BLOCKING

If ANY core quality gate fails, the agent MUST NOT produce the session report marked as complete. Instead it must fix the issue, re-run the verification, and only then finalise the report. This is the most important difference from vibe coding — the agent is responsible for correctness.

SCAFFOLDING REFERENCE — WHAT THE AGENT GENERATES ON FIRST RUN

09

When `src/` is empty (or contains only a `.gitkeep`), the agent runs step 08 and generates this minimum viable scaffold. Stack-specific files are shown for the three most common choices.

TypeScript / Node.js

```
# Node.js / TypeScript scaffold
src/
  index.ts           # Entry point, graceful shutdown on SIGTERM
  app.ts             # Express/Fastify app factory
  routes/health.ts   # GET /health -> { status, version, uptime }
  middleware/
    auth.ts          # JWT or API-key middleware
    logger.ts        # Pino structured JSON – includes trace-id
    errorHandler.ts  # Global error -> RFC 7807 Problem Details
  config/env.ts       # zod-validated env vars, fail fast on missing
  types/index.ts      # Shared domain types
  tests/health.test.ts # Supertest smoke test
  package.json        # Exact version pins (no ^ or ~)
  tsconfig.json       # strict: true
  .eslintrc.json      .prettierrc Dockerfile docker-compose.yml
  .github/workflows/ci.yml # lint -> build -> test -> docker build
```

Python / FastAPI


```
# Python / FastAPI scaffold
src/
  main.py          # FastAPI app, lifespan context manager
  routers/health.py # GET /health -> { status, version }
  middleware/logging.py # structlog middleware with trace-id
  config/settings.py # Pydantic BaseSettings, env-driven
  models/base.py    # Shared Pydantic response models
tests/test_health.py # httpx + pytest smoke test
pyproject.toml      # Exact deps via uv or poetry
ruff.toml            # Linting + formatting
Dockerfile docker-compose.yml .github/workflows/ci.yml
# ci: ruff -> pytest -> docker build
```

TEMPLATE REFERENCE — ADR FORMAT

10

Place this template at templates/ADR.md.tpl. The agent fills it in for every new decision.

```
# ADR-[NNN]: [Short title of decision]

Date: [YYYY-MM-DD]
Status: [Proposed | Accepted | Deprecated | Superseded by ADR-NNN]
Deciders: [AI Agent | Human Architect | Team]

## Context
[What is the situation that requires a decision?
What constraints exist? What alternatives were considered?]

## Decision
[What was decided? Be specific — name the library, pattern, or tool.]

## Consequences
Positive: [Benefits of this decision]
Negative: [Trade-offs or risks introduced]
Neutral: [Other effects]

## Compliance
[ ] Requirements in requirements/04-constraints.md are satisfied
[ ] No conflicting decision exists in a prior ADR
```

COPILOT INSTRUCTIONS FILE — .github/copilot-instructions.md

11

Commit this file to your repository. GitHub Copilot reads it automatically at the start of every Chat session when you are working inside this repository. It makes Copilot requirements-aware without you needing to attach the PDF every time.

```
# GitHub Copilot Instructions
# This file is read automatically by GitHub Copilot in every Chat session.

## Project context
This is an AI-assisted microservice development project.
The repository follows the AI-MSF PoC framework.

## Mandatory reading before any code action
ALWAYS read the following files before writing or modifying code:
1. .ai-state.md          - current session state and agent memory
2. requirements/*.md     - all requirement files, in numeric order
3. docs/architecture/overview.md - architectural decisions
4. docs/adr/*.md         - all Architecture Decision Records

## Rules
- Treat requirements/*.md as the authoritative source of truth.
- Never hardcode secrets. Use environment variables.
- Every endpoint must be documented in docs/api/openapi.yaml.
- Every architectural decision must produce an ADR.
- Update .ai-state.md at the end of every session.
- Run the quality gates checklist (Section 08 of requirements PDF)
  before marking any increment as complete.

## Skills
Load skills/*.md files relevant to the current task.
Do not load all skills — only those matching the work being done.
```

INDUSTRY CONTEXT — HOW THE GIANTS SOLVE GREENFIELD (JUNE 2026)

12

GitHub Spec Kit (Microsoft, open source)

Agent-agnostic CLI. Bootstraps constitution -> specify -> plan -> tasks -> implement workflow. 90k+ stars. Supports Copilot, Claude Code, Gemini CLI, Cursor, Windsurf. Gap: brownfield requires manual customisation; no doc auto-sync post-implementation.

AWS Kiro (preview)

Standalone IDE (Code OSS fork). Spec mode -> design -> tasks -> implementation. Agent hooks enforce consistency on file-system events. Autopilot mode available. Gap: standalone IDE required; specs can drift from code post-implementation.

Azure AI Foundry (Build 2026)

Hosted agents with memory, toolboxes, tracing, governance. Semantic Kernel + AutoGen merged. Work IQ APIs give agents email, calendar, org context. Rayfin SDK for backend scaffolding. Gap: Azure-native; high vendor lock-in; separate concern from doc synchronisation.

This PoC — what's different

The repo IS the agent's memory. Plain markdown, zero vendor lock-in. Unique: stack detection from specs, doc sync post-implementation, idempotent resumption, self-review quality gate, .ai-state.md persistent memory. Works with any AI coding agent.

THE SHIFT (2026 DATA)

46% of all code written by active developers is now AI-generated (June 2026).

Development is shifting from writing code to writing specifications — developers author intent, agents manifest it. The architect role has never been more valuable. This PoC establishes the core pattern that can later scale to multi-agent orchestration and enterprise governance.

AI-Assisted Microservice Factory PoC | Requirements & Agent Runbook | v1.0 | June 2026 | Compatible with: GitHub Copilot Agent, Claude Code, Gemini CLI, AWS Kiro, Cursor, Windsurf